

下面是**第一节线上课 2 小时课案**，按你现在的无边记设计来写。整体风格是你可以直接照着讲，里面包含：

1. 每个时间段讲什么
2. 你可以直接说的话
3. 无边记里应该放的关键笔记
4. 互动问题
5. 标准答案
6. 如果讲快/讲慢怎么调整

这一节课主题：

第 1 节：项目整体理解与数据流

一、课前准备

上课前你需要准备好这些东西：

1. 无边记课件
2. paper_keyboard_v2 项目目录
3. 一个可运行的 replay_session_demo
4. 一个 example session JSON
5. 一个 keyboard_number_v2 layout JSON
6. 如果条件允许，准备一次完整 demo 演示

如果完整实时 demo 不稳定，也没关系。第一节可以只演示 replay，因为这一节课的主目标不是调传感器，而是让学生理解：

真实输入最后会变成 JSON
InputEngine 先处理 JSON
后面线下再接回摄像头和麦克风

二、第一节总目标

你可以在无边记第一块写：

第 1 节课：项目整体理解与数据流

今天要弄明白三件事：

1. 这个纸面键盘项目想实现什么？

2. 真实世界的敲击会变成什么数据？
3. layout、session、InputEngine 分别负责什么？

你开场可以直接说：

今天我们不会一上来就写复杂代码。我们先要看懂这个项目是怎么工作的。这个项目表面上看是“在纸上打字”，但从程序角度看，其实是：把真实世界里的手指敲击，变成一帧一帧的数据，再让程序根据数据判断用户按了哪个键。

然后告诉学生：

今天结束时，你至少要能说清楚三句话：layout 记录按键在哪里。session 记录每一帧发生了什么。InputEngine 根据 frame 和 layout 判断是否输出字符。

三、完整时间安排

推荐标准流程：

0:00 - 0:10	开场：项目目标 + 学生猜想
0:10 - 0:25	最终效果演示 + replay 演示
0:25 - 0:45	项目整体数据流
0:45 - 1:10	session JSON
1:10 - 1:35	layout JSON
1:35 - 1:50	InputEngine + Python 最小基础
1:50 - 2:00	项目日志 + 总结

如果学生慢，就砍掉 Python 小练习和备用互动。如果学生快，就加“点在不在矩形里”“设计一个 key”“修改 JSON 后猜输出”。

四、逐分钟课案

0:00 - 0:05 开场：今天做什么

你可以直接说：

我们这个项目叫 Paper Keyboard，也就是纸面键盘。最终目标是：用户在纸上敲一个按键，电脑能判断出他按了哪个键，并输出对应字符。但是电脑本身看不懂“我按了纸上的 1”。所以我们需要把真实世界里的动作，转换成程序能处理的数据。

无边记写：

电脑不能直接理解：
“我按了纸上的哪个键”

我们要做的是：
真实动作 → 数据 → 判断 → 输出

然后说：

今天我们先不急着写完整程序。我们先弄清楚这个系统里每一部分负责什么。

0:05 - 0:10 学生猜想：电脑需要知道什么？

无边记问题：

如果让电脑判断“我按了哪个纸面按键”，它至少需要知道什么？

你问学生：

你想一下，如果电脑要判断你按了哪个键，它至少得知道哪些信息？

引导答案：

1. 手指在哪里
2. 有没有真的敲下去
3. 每个按键在哪里

如果学生答不出来，你可以逐步问：

如果电脑不知道你的手指在哪里，它能判断吗？如果你只是把手指放在键上，没有敲，它要不要输出？
如果电脑不知道纸上每个键的位置，它能知道你按的是 1 还是 2 吗？

无边记最终补全：

手指在哪里 → `fingers` 里的 `x, y`
有没有敲击 → `tap.audio`
每个按键在哪里 → `layout JSON`

标准答案：

电脑至少需要知道：

1. 手指坐标
2. 是否发生敲击
3. 按键布局

0:10 - 0:18 演示 1：最终效果

如果实时 demo 可用，你就演示纸面键盘。如果实时 demo 不稳定，就用视频/口头说明 + replay 演示替代。

你可以说：

先看最终我们想做出来的效果。我把手指放到纸面键盘上，敲一下，电脑会输出对应字符。注意看，这里面其实有两个动作：第一，手指要在某个键上。第二，要真的敲下去。

无边记写：

演示观察：

1. 手指移动到纸面按键上
2. 手指敲击纸面
3. 程序输出字符

问题：

电脑是怎么知道我按了哪个键的？

互动问题：

只把手指放在键上，不敲，要不要输出？

标准答案：

不要。因为只是停在键上，不代表真正输入。

0:18 - 0:25 演示 2：session replay

你切到 replay demo。

你可以说：

刚才那个是实时系统，需要摄像头和麦克风。但是我们线上课先不直接处理摄像头和麦克风，因为那样太复杂。我们先把真实输入保存成一个 JSON 文件。之后程序只要读取这个 JSON，就可以回放刚才发生过的输入过程。

运行：

```
python app/replay_session_demo.py
```

你边运行边说：

现在没有实时摄像头，也没有实时麦克风。但是程序还是能输出结果。原因是 JSON 里已经记录了每一帧发生了什么。

无边记写：

session replay:

不使用实时摄像头
不使用实时麦克风
只读取 JSON 文件
程序仍然可以输出字符

原因：
JSON 已经记录了输入过程

互动问题：

为什么不开摄像头和麦克风，程序还能输出？

标准答案：

因为 JSON 文件里已经保存了手指位置和敲击信息，程序可以直接读取这些数据来判断。

0:25 - 0:35 项目整体数据流

无边记写：

真实世界：
用户在纸面键盘上敲击

摄像头：
找到手指在纸面上的位置

麦克风：
判断有没有敲击声音

session JSON：
把每一帧的数据保存下来

layout JSON：
记录每个按键在纸上的位置

InputEngine：
根据 frame 和 layout 判断是否输出字符

你可以这样讲：

这个项目可以拆成几个部分。摄像头负责看手指在哪里。麦克风负责听有没有敲击。session JSON 负责记录每一帧发生了什么。layout JSON 负责记录每个按键在哪里。InputEngine 是真正做判断的地方。

然后强调：

摄像头不直接告诉我们“按了 1”。麦克风也不直接告诉我们“按了 1”。摄像头只提供手指位置。麦克风只提供有没有敲击。真正判断按了哪个键的是 InputEngine。

无边记重点句：

摄像头不直接判断按键。
麦克风不直接判断按键。
真正做判断的是 InputEngine。

0:35 - 0:45 数据流互动检查

无边记问题：

问题 1：
如果我要知道“手指在哪里”，看哪个部分？

问题 2：
如果我要知道“按键 1 在哪里”，看哪个部分？

问题 3：
如果我要判断“要不要输出字符”，看哪个部分？

你让学生回答。

标准答案：

问题 1: session JSON 里的 fingers / x / y
问题 2: layout JSON
问题 3: InputEngine

如果学生答得慢，你就重复三句话：

layout = 按键在哪里
session = 每一帧发生了什么
InputEngine = 判断是否输出

如果学生答得快，加一个追问：

如果程序输出错了，可能是哪一部分出了问题？

标准答案可以接受这些：

1. 摄像头识别的手指位置错了
2. 麦克风误判了敲击
3. layout 里按键位置不对
4. InputEngine 判断逻辑写错了

你可以总结：

这就是工程调试的思路。程序错了，不要只说“代码错了”，而是要判断错误可能来自数据流的哪一段。

五、session JSON 部分

0:45 - 0:50 引入 session

你可以说：

接下来我们先看 session JSON。session 可以理解成“一次输入过程的记录”。比如我敲了 1、2、3，这个过程就可以保存成一个 session 文件。

无边记写：

session JSON：
记录一次输入过程

一个 session 里有很多 frame
每个 frame 表示某一个时刻发生了什么

然后说：

我们的程序不是直接处理“我按了 123”这句话。它处理的是一帧一帧的数据。

0:50 - 0:58 讲 frame 字段

无边记放简化版字段，不要一上来放完整嵌套 JSON：

一个 frame 里最重要的信息：

```
frame_id: 2  
t: 0.033
```

```
fingers:
  finger_id: 1
  x: 106.0
  y: 82.8

tap:
  audio: true
```

你逐个解释：

```
frame_id: 第几帧
t: 这一帧发生的时间
fingers: 这一帧看到的手指
finger_id: 手指编号
x, y: 手指在纸面上的位置
tap.audio: 这一帧有没有检测到敲击声
```

你可以说：

x 和 y 不是屏幕坐标，而是纸面上的坐标。我们最终希望 InputEngine 不关心摄像头画面，只关心纸面坐标。所以摄像头那部分以后会负责把图像里的手指位置转换成纸面上的 x、y。

0:58 - 1:03 解释 finger_id = 1

无边记写：

当前版本简化：

```
每一帧最多只有一个 finger
finger_id 固定为 1
1 代表食指
```

注意：
这不是系统现在已经能识别出食指
而是我们当前阶段的人为约定

你可以这样讲：

现在我们还没有做双摄像头，也没有做颜色贴纸识别，所以系统暂时不能真正区分每根手指。为了先把主流程做通，我们先假设只有一个手指，也就是食指。所以 finger_id 固定为 1。以后如果我们能识别多个手指，这里就可以出现 finger_id = 1、2、3 等不同编号。

互动问题：

```
finger_id = 1 是什么意思？
```

标准答案：

当前版本里，`finger_id = 1` 表示我们假设正在使用的那个食指。
它不是系统真实识别出来的结果，而是当前阶段的简化约定。

1:03 - 1:10 session 互动练习

无边记放 3 个小问题。

问题 1：

如果 `tap.audio = false`，说明什么？

答案：

这一帧没有检测到敲击声。

问题 2：

如果 `tap.audio = true`，说明什么？

答案：

这一帧检测到了敲击声。

问题 3：

如果 `fingers` 是空列表，说明什么？

答案：

这一帧没有检测到手指。

再加一个判断题：

手指在某个键上，但是 `audio=false`，要不要输出？

答案：

不要输出。因为没有检测到敲击。

你可以总结：

`session JSON` 告诉我们每一帧发生了什么。但是它还没有直接告诉我们“按了哪个键”。要知道按了哪个键，还需要 `layout`。

六、layout JSON 部分

1:10 - 1:15 引入 layout

你可以说：

现在我们知道手指在哪里了。但是光知道手指坐标还不够。比如手指在 `x=25, y=30`，电脑并不知道这是按键 1、按键 2，还是空白区域。所以我们还需要 `layout`。`layout` 记录每个按键在纸面上的位置。

无边记写：

`layout JSON`：
记录纸面键盘上每个按键的位置

`session` 告诉我们：手指在哪里
`layout` 告诉我们：每个键在哪里

1:15 - 1:23 讲 key 的字段

无边记放：

一个 `key` 的例子：

```
label: "1"  
x: 20  
y: 20  
w: 20  
h: 22
```

解释：

```
label: 这个键输出什么字符
x: 按键左上角的 x 坐标
y: 按键左上角的 y 坐标
w: 按键宽度
h: 按键高度
```

你可以说：

每个按键都可以看作一个矩形。x 和 y 是左上角的位置。w 是宽度，h 是高度。所以这个按键覆盖的范围是：x 从 20 到 40，y 从 20 到 42。

无边记写：

```
key 1 的范围：

x: 20 到 40
y: 20 到 42
```

1:23 - 1:30 点在不在矩形里

无边记写：

```
判断一个 finger 是否在 key 里面：

finger_x >= key_x
finger_x <= key_x + key_w
finger_y >= key_y
finger_y <= key_y + key_h
```

你可以说：

这一步不是 AI，也不是很复杂的算法。它本质上就是判断一个点有没有落在一个矩形范围里。我们后面写 InputEngine 的时候，最核心的一步就是这个判断。

互动题：

```
key 1:
x = 20
y = 20
w = 20
h = 22

点 A: x=25, y=30
点 B: x=45, y=30
点 C: x=25, y=50
```

它们在不在 key 1 里？

标准答案：

A 在。
因为 25 在 20 到 40 之间，30 在 20 到 42 之间。

B 不在。
因为 $x=45$ 超出了 20 到 40 的范围。

C 不在。
因为 $y=50$ 超出了 20 到 42 的范围。

1:30 - 1:35 layout 总结

你可以问：

如果我想让键变大，应该改什么？

标准答案：

改 w 和 h。

继续问：

如果我想把键往右移，应该改什么？

标准答案：

改 x，让 x 变大。

继续问：

如果我想把键往下移，应该改什么？

标准答案：

改 y ，让 y 变大。

你可以总结：

layout 不是图片，它是数据。我们修改 layout 数据，就可以改变程序认为按键所在的位置。

无边记重点句：

layout 不是键盘图片。
layout 是按键位置数据。

七、InputEngine 部分

1:35 - 1:42 InputEngine 是什么

无边记写：

InputEngine：系统的大脑

它不直接处理摄像头画面。
它不直接处理麦克风声音。

它只读取：

1. frame 数据
2. layout 数据

然后判断：

这一帧要不要输出一个按键？

你可以说：

InputEngine 是我们线上课最核心要写的部分。它不需要知道摄像头怎么识别手指，也不需要知道麦克风怎么检测声音。它只需要处理整理好的 frame 数据。这样我们就可以先把核心判断逻辑学明白。

1:42 - 1:47 有效输入的两个条件

无边记写：

一次有效输入需要两个条件：

条件 1：

手指在某个 key 里面

条件 2：
tap.audio 是 true

你可以讲：

这两个条件缺一不可。如果手指在 key 上，但是没有敲击，不输出。如果有敲击，但是手指不在任何 key 上，也不输出。只有手指在某个 key 里，并且 audio=true，才输出这个 key。

互动题：

情况 1：
手指在 key 1 上，audio=false
会不会输出？

情况 2：
手指在空白区域，audio=true
会不会输出？

情况 3：
手指在 key 1 上，audio=true
会不会输出？

标准答案：

情况 1：不输出。
原因：没有检测到敲击。

情况 2：不输出。
原因：手指不在任何按键上。

情况 3：输出 key 1。
原因：手指在 key 上，并且检测到敲击。

1:47 - 1:50 预告防重复

如果时间正常，只简单预告。

你可以说：

后面我们还会遇到一个问题：如果敲击声音连续几帧都是 true，会不会重复输出好几次？这个问题我们之后会用一个简单的状态变量解决。今天先知道 InputEngine 的基本判断条件就可以。

无边记写：

后续问题：
连续多帧 audio=true 可能导致重复输出

之后解决：
加入一个简单状态变量，等待 audio 变回 false

这里不要展开太多，否则第一节课内容会过重。

八、Python 最小基础部分

1:50 - 1:55 dict / list 快速过一下

如果学生特别弱，这部分可以多讲。如果学生已经理解，就快速带过。

无边记写：

今天需要知道的 Python 基础：

dict: 用名字取数据
list: 用编号取数据
True / False: 表示真假
print: 输出结果

例子：

```
frame["frame_id"]  
frame["tap"]["audio"]  
frame["fingers"][0]  
finger["x"]  
finger["y"]
```

你可以讲：

JSON 读进 Python 以后，基本上就是 dict 和 list。dict 用名字取数据，比如 frame["tap"]。list 用编号取数据，比如 fingers[0]。因为现在我们假设只有一个 finger，所以先取 fingers[0]。

互动问题：

frame["tap"]["audio"] 取到的是什么？

答案：

这一帧有没有检测到敲击声，结果是 true 或 false。

问题：

```
frame["fingers"][0] 取到的是什么？
```

答案：

这一帧里的第一个 finger。当前版本里，也就是我们假设的食指。

九、项目日志和总结

1:55 - 2:00 项目日志

无边记放：

第 1 节项目日志要回答的问题：

1. 这个项目最终想实现什么？
2. 电脑为什么不能直接知道“我按了哪个纸面按键”？
3. 摄像头负责提供什么信息？
4. 麦克风负责提供什么信息？
5. session JSON 记录什么？
6. layout JSON 记录什么？
7. InputEngine 负责什么？
8. 为什么当前版本先假设 `finger_id = 1`？

你可以说：

每节课结束前，我们都会写一点项目日志。这个日志不是作文，而是帮你检查自己有没有真的理解项目。第一节课最重要的就是能把这几个问题说清楚。

课堂上带他填这几个短句：

```
layout = _____  
session = _____  
InputEngine = _____  
tap.audio = _____  
finger_id = 1 表示 _____
```

标准答案：

```
layout = 按键在哪里  
session = 每一帧发生了什么  
InputEngine = 判断是否输出字符  
tap.audio = 有没有检测到敲击  
finger_id = 1 表示当前版本假设使用的食指
```

最后总结：

今天我们没有写很多代码，但我们把项目最重要的数据流讲清楚了。后面写代码的时候，所有代码其实都是围绕这句话展开：读取一帧 frame，看手指坐标在哪个 key 里，再看 audio 是不是 true，最后决定要不要输出字符。

十、讲快时的备用内容

如果 1:35 左右就讲完了 session 和 layout，可以加这些任务。

备用任务 1：看 frame 猜含义

给学生：

```
frame_id: 5  
finger_id: 1  
x: 35  
y: 28  
tap.audio: false
```

问题：

1. 这是第几帧？
2. 手指编号是多少？
3. 手指坐标是多少？
4. 有没有检测到敲击？
5. 这一帧会不会输出？

答案：

1. 第 5 帧
2. finger_id 是 1
3. x=35, y=28
4. 没有检测到敲击
5. 不会输出，因为 audio=false

备用任务 2：设计一个 key

让学生设计：

```
label:  
x:  
y:  
w:  
h:
```

然后问：

点 $x=\text{多少}$, $y=\text{多少}$ 会在这个 key 里?
点 $x=\text{多少}$, $y=\text{多少}$ 会不在这个 key 里?

标准检查方式：

x 是否在 key_x 到 $\text{key_x} + \text{key_w}$ 之间
 y 是否在 key_y 到 $\text{key_y} + \text{key_h}$ 之间

备用任务 3：判断是否输出

给三条：

A: 手指在 key 2 上, $\text{audio}=\text{false}$
B: 手指在空白区域, $\text{audio}=\text{true}$
C: 手指在 key 2 上, $\text{audio}=\text{true}$

答案：

A 不输出
B 不输出
C 输出 key 2

备用任务 4：预告下一节课函数

无边记写：

下节课要写的核心函数：

输入：

finger_x
finger_y
keys

输出：

这个点在哪个 key 里面

你问学生：

这个函数应该做什么？

标准答案：

它应该检查每一个 key，看 finger_x 和 finger_y 是否落在这个 key 的矩形范围里。如果在，就返回这个 key。

十一、讲慢时可以砍掉什么

如果学生理解比较慢，你可以砍掉：

1. Python dict/list 细节
2. 备用练习
3. 防重复问题预告
4. 修改 layout 的延伸问题

必须保留：

1. 项目目标
2. 摄像头 / 麦克风 / JSON / InputEngine 分工
3. session JSON 的 fingers 和 tap.audio
4. layout JSON 的 x, y, w, h
5. 有效输入的两个条件

只要学生能说清楚下面这些，就算第一节课成功：

session 记录每一帧发生了什么
layout 记录按键在哪里
InputEngine 判断是否输出字符
tap.audio 表示有没有敲击
finger_id=1 是当前版本的人为简化

十二、第一节课结束时的检查题和答案

你可以最后快速问一遍。

问题 1

这个项目最终想实现什么？

答案：

让用户在纸面键盘上敲击按键，系统能够判断按下的是哪个键，并输出对应字符。

问题 2

摄像头负责什么？

答案：

摄像头负责提供手指在纸面上的位置。

问题 3

麦克风负责什么？

答案：

麦克风负责检测有没有敲击声音。

问题 4

session JSON 记录什么？

答案：

session JSON 记录一次输入过程中的每一帧数据，包括手指位置和是否检测到敲击。

问题 5

layout JSON 记录什么？

答案：

layout JSON 记录纸面键盘上每个按键的位置和大小。

问题 6

InputEngine 负责什么？

答案：

InputEngine 负责根据 frame 数据和 layout 数据判断这一帧是否应该输出某个按键。

问题 7

tap.audio=true 代表什么？

答案：

代表这一帧检测到了敲击声。

问题 8

`finger_id=1` 代表什么？

答案：

当前版本中，`finger_id=1` 代表我们假设使用的食指。它是人为简化约定，不是现在已经真正完成了多指识别。

问题 9

一次有效输入需要哪两个条件？

答案：

1. 手指在某个 `key` 的范围内。
2. `tap.audio` 是 `true`。

问题 10

为什么先用 `JSON`，而不是一开始就处理摄像头和麦克风？

答案：

因为真实传感器比较复杂，可能有识别错误和噪声。先用 `JSON` 可以把问题简化，让我们先专心学习核心判断逻辑。

十三、第一节课无边记核心笔记汇总

你可以直接把这些作为无边记里的重点卡片。

电脑不能直接理解：
“我按了纸上的哪个键”

系统至少需要知道：
1. 手指在哪里

2. 有没有敲击
3. 每个按键在哪里

摄像头：
提供手指位置

麦克风：
提供敲击信息

session JSON：
记录每一帧发生了什么

layout JSON：
记录每个按键在哪里

InputEngine：
判断是否输出字符

session 里的一个 frame 主要包含：

frame_id: 第几帧
t: 时间
fingers: 这一帧看到的手指
finger_id: 手指编号
x, y: 手指在纸面上的位置
tap.audio: 有没有检测到敲击

当前版本简化：

只有一个 finger
finger_id 固定为 1
1 代表食指

之后可以扩展：
双摄像头 / 颜色贴纸 → 多个 finger_id

layout 里的一个 key 主要包含：

label: 按键输出字符
x: 按键左上角 x 坐标
y: 按键左上角 y 坐标
w: 按键宽度
h: 按键高度

判断 finger 是否在 key 里：

finger_x 在 key_x 到 key_x + key_w 之间
finger_y 在 key_y 到 key_y + key_h 之间

一次有效输入：

手指在某个 key 里面
并且
`tap.audio = true`

三种情况：

手指在 key 上, `audio=false` → 不输出
手指在空白处, `audio=true` → 不输出
手指在 key 上, `audio=true` → 输出

十四、课后小作业

建议第一节作业不要难。

可以布置：

作业 1：
完成 `lesson_01_project_log.md`

作业 2：
回答下面 5 个问题：

1. session JSON 和 layout JSON 有什么区别？
2. `tap.audio` 是做什么用的？
3. 为什么 `finger_id` 现在固定为 1？
4. 如果手指在 key 上但 `audio=false`，程序要不要输出？
5. 如果 `audio=true` 但手指在空白处，程序要不要输出？

标准答案：

1. session 记录每一帧发生了什么，layout 记录每个按键在哪里。
2. `tap.audio` 表示这一帧有没有检测到敲击声。
3. 因为当前版本先假设只有一个手指，也就是食指。

4. 不输出，因为没有检测到敲击。
5. 不输出，因为手指不在任何按键上。

这一节课的重点不是让学生写很多代码，而是让他真正建立这个工程的基本模型。下节课开始写 `find_key_at_position()`，学生就会知道这段代码为什么存在。